

Module 5: Transport Layer

Computer Networks

Q1. Explain the functions of the transport layer.

The transport layer (Layer 4 of the OSI model) is responsible for end-to-end communication between processes on different hosts. Its key functions are:

- **Segmentation and Reassembly:** Divides the message from the application layer into smaller segments at the sender side, and reassembles them at the receiver side.
- **Connection Control:** Provides either connection-oriented (TCP) or connectionless (UDP) communication between hosts.
- **Flow Control:** Ensures the sender does not overwhelm the receiver by controlling the rate of data transmission.
- **Error Control:** Detects and retransmits lost or corrupted segments to ensure reliable data delivery.
- **Congestion Control:** Monitors and reduces network congestion by limiting the amount of data sent into the network.
- **Port Addressing:** Uses port numbers to identify specific processes/applications on a host, enabling multiplexing and demultiplexing.
- **Multiplexing / Demultiplexing:** Allows multiple applications to share the same network connection by tagging data with port numbers.

Q2. Explain TCP's 3-way handshake.

The **TCP 3-Way Handshake** is used to establish a reliable connection between a client and a server before data transfer begins.

Steps:

1. **SYN (Synchronize):** The client sends a TCP segment with the SYN flag set and an initial sequence number (ISN) to the server, requesting a connection.
2. **SYN-ACK (Synchronize-Acknowledge):** The server responds with a segment having both SYN and ACK flags set. It acknowledges the client's ISN and sends its own ISN.
3. **ACK (Acknowledge):** The client sends an ACK segment acknowledging the server's ISN. The connection is now established and data transfer can begin.

Summary:

Client $\xrightarrow{\text{SYN}}$ Server $\xrightarrow{\text{SYN-ACK}}$ Client $\xrightarrow{\text{ACK}}$ Server

This ensures both sides are ready to communicate and agree on initial sequence numbers.

Q3. Explain TCP's 4-way connection termination.

TCP uses a **4-way handshake** to gracefully terminate a connection, because each direction of the connection is closed independently.

Steps:

1. **FIN from Client:** The client sends a **FIN** segment, indicating it has finished sending data.
2. **ACK from Server:** The server acknowledges the client's **FIN** with an **ACK**. The server may still send remaining data to the client (half-close state).
3. **FIN from Server:** Once the server is also done sending, it sends its own **FIN** segment to the client.
4. **ACK from Client:** The client sends a final **ACK** to acknowledge the server's **FIN**. After a *TIME_WAIT* period, the connection is fully closed.

Summary:

Client $\xrightarrow{\text{FIN}}$ Server $\xrightarrow{\text{ACK}}$ Client, Server $\xrightarrow{\text{FIN}}$ Client $\xrightarrow{\text{ACK}}$ Server

The *TIME_WAIT* state ensures that any delayed packets are discarded before the connection is closed completely.

Q4. Compare TCP and UDP.

Feature	TCP	UDP
Full Form	Transmission Control Protocol	User Datagram Protocol
Connection	Connection-oriented	Connectionless
Reliability	Reliable (guaranteed delivery)	Unreliable (no guarantee)
Error Control	Yes (retransmission)	No
Flow Control	Yes	No
Speed	Slower (overhead)	Faster
Order of Data	Maintained	Not guaranteed
Header Size	20–60 bytes	8 bytes
Use Cases	HTTP, FTP, Email, SSH	DNS, VoIP, Video streaming, Gaming

Q5. What is port addressing? Explain with examples.

Port addressing is the mechanism used by the transport layer to identify a specific process or application on a host. While IP addresses identify the host, port numbers identify the application running on that host.

- Port numbers are **16-bit integers** ranging from **0 to 65535**.
- They are divided into three categories:
 - **Well-known ports (0–1023):** Reserved for standard services.

- **Registered ports (1024–49151):** Used by specific applications.
- **Dynamic/Private ports (49152–65535):** Assigned temporarily by the OS.

Examples of well-known port numbers:

Protocol	Port Number	Usage
HTTP	80	Web browsing
HTTPS	443	Secure web browsing
FTP	21	File transfer
SSH	22	Secure shell
DNS	53	Domain name resolution
SMTP	25	Email sending

Example: When you open a website, your browser uses destination port 80 (HTTP). Your OS assigns a temporary source port (e.g., 54321) so the server knows where to send the response.

Q6. Explain leaky bucket congestion control.

The **Leaky Bucket algorithm** is a congestion control and traffic shaping mechanism that regulates data transmission at a constant rate regardless of burst input.

How it works:

- Incoming data packets are placed into a **bucket (buffer)**.
- The bucket leaks (sends out) data at a **fixed constant rate**.
- If packets arrive faster than the leak rate, the excess is queued.
- If the **bucket overflows** (buffer is full), incoming packets are **discarded**.

Key characteristics:

- Smooths out **bursty traffic** into a steady stream.
- Output rate is always constant, regardless of input rate.
- Does not allow any burst to pass through.

Example: If packets arrive at 10 Mbps but the bucket leaks at 2 Mbps, the extra packets queue up. If the queue fills, packets are dropped.

Limitation: It cannot handle sudden bursts, even if the network is temporarily idle.

Q7. Explain token bucket congestion control.

The **Token Bucket algorithm** is an improvement over the leaky bucket that allows **controlled bursts** of traffic while still enforcing an average rate limit.

How it works:

- Tokens are generated at a fixed rate and placed into a **token bucket**.
- Each packet that is to be sent must consume one (or more) tokens from the bucket.

- If tokens are available, the packet is transmitted immediately.
- If no tokens are available, the packet must **wait** until tokens are generated.
- The bucket has a **maximum capacity** — excess tokens are discarded.

Key characteristics:

- Allows **bursts** up to the bucket size when tokens have accumulated.
- Enforces an **average rate** equal to the token generation rate.
- More flexible than leaky bucket.

Comparison with Leaky Bucket:

Feature	Leaky Bucket	Token Bucket
Output Rate	Always constant	Variable (allows bursts)
Burst Handling	Not allowed	Allowed up to bucket size
Flexibility	Low	High

Example: If tokens accumulate at 2 tokens/sec and the bucket holds 10 tokens, a burst of 10 packets can be sent at once when the bucket is full.